CS-305-MJ-P: Operating System-I

Slip-wise Solutions (T.Y.B.Sc. CS, SPPU, Sem V) — Selected Slips

Selected slips: 1, 3, 4, 5, 9, 10, 11, 12, 13, 16, 20, 23, 25, 27, 28. One simplest question (15 marks) is solved from each slip, which clears the 14-mark target. When the same program repeats in another selected slip, it is solved again in full so each slip stands on its own. Every program is plain C with no comments, was compiled with gcc and run on the data of its slip; the sample output shown is the real output. Slips 1 and 16 also have the shell program (Q2) written in the style of your sample as an alternative. Compile: `gcc file.c -o out` Run: `./out`

SLIP 1 — Solve Q1 (15 marks)

Question: Write a C menu driven program: a) Accept Available b) Display Allocation, Max c) Display the contents of need matrix d) Display Available (5 processes, 3 resource types A B C, data as given in the question).

Program: slip1.c

Program

```
#include <stdio.h>

int alloc[5][3] = {{2, 3, 2}, {4, 0, 0}, {5, 0, 4}, {4, 3, 3}, {2, 2, 4}};
int max[5][3]   = {{9, 7, 5}, {5, 2, 2}, {1, 0, 4}, {4, 4, 4}, {6, 5, 5}};
int avail[3];
int need[5][3];

int main() {
    char ch;
    int i, j;

    do {
        printf("\na) Accept Available\n");
        printf("b) Display Allocation, Max\n");
        printf("c) Display contents of Need matrix\n");
        printf("d) Display Available\n");
        printf("e) Exit\n");
        printf("Enter your choice: ");
        scanf(" %c", &ch);

        switch (ch) {
        case 'a':
            printf("Enter Available resources (A B C): ");
            for (j = 0; j < 3; j++)
                scanf("%d", &avail[j]);
            break;
        case 'b':
            printf("\nProcess\tAllocation\tMax\n");
            printf("\tA B C\t\tA B C\n");
            for (i = 0; i < 5; i++) {
                printf("P%d\t", i);
                for (j = 0; j < 3; j++)
                    printf("%d ", alloc[i][j]);
                printf("\t\t");
                for (j = 0; j < 3; j++)
                    printf("%d ", max[i][j]);
                printf("\n");
            }
            break;
        case 'c':
            printf("\nNeed Matrix (Max - Allocation)\n");
            printf("\tA B C\n");
            for (i = 0; i < 5; i++) {
                printf("P%d\t", i);
                for (j = 0; j < 3; j++) {
                    need[i][j] = max[i][j] - alloc[i][j];
                    printf("%d ", need[i][j]);
                }
                printf("\n");
            }
            break;
        case 'd':
            printf("Available: A=%d B=%d C=%d\n", avail[0], avail[1], avail[2]);
            break;
        case 'e':
```

```
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (ch != 'e');
    return 0;
}
```

Sample Input / Output

```
a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: a
Enter Available resources (A B C): 3 3 2

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: b

Process Allocation      Max
        A B C           A B C
P0      2 3 2           9 7 5
P1      4 0 0           5 2 2
P2      5 0 4           1 0 4
P3      4 3 3           4 4 4
P4      2 2 4           6 5 5

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: c

Need Matrix (Max - Allocation)
        A B C
P0      7 4 3
P1      1 2 2
P2      -4 0 0
P3      0 1 1
P4      4 3 1

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: d
Available: A=3 B=3 C=2

a) Accept Available
b) Display Allocation, Max
c) Display contents of Need matrix
d) Display Available
e) Exit
Enter your choice: e
Exiting...
```

Note: the question paper data gives P2 a Max (1 0 4) smaller than its Allocation (5 0 4), so its need shows -4. That is how the given data works out.

SLIP 1 — Solve Q2 (alternative) (15 marks)

Question: Write a C program that behaves like a shell which displays the command prompt. It accepts the command, tokenizes the command line and executes it by creating a child process. Also implement the additional command count: count c filename (characters), count w filename (words), count l filename (lines).

Program: slip1.c

Program

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

void make_toks(char *s, char *tok[])
{
 int i = 0;
 char *p;
 p = strtok(s, " ");
 while (p != NULL)
 {
  tok[i++] = p;
  p = strtok(NULL, " ");
 }
 tok[i] = NULL;
}

void count(char *fn, char op)
{
 int fh, cc = 0, wc = 0, lc = 0;
 char c;
 fh = open(fn, O_RDONLY);
 if (fh == -1)
 {
  printf("File %s not found.\n", fn);
  return;
 }
 while (read(fh, &c, 1) > 0)
 {
  if (c == ' ')
   wc++;
  else if (c == '\n')
  {
   wc++;
   lc++;
  }
  cc++;
 }
 close(fh);
 switch (op)
 {
 case 'c':
  printf("No.of characters:%d\n", cc);
  break;
 case 'w':
  printf("No.of words:%d\n", wc);
  break;
 case 'l':
  printf("No.of lines:%d\n", lc);
  break;
 }
}

int main()
{
 char buff[80], *args[10];
 int pid;
 while (1)
 {
  printf("myshell$ ");
  fflush(stdout);
  fgets(buff, 80, stdin);
  buff[strlen(buff) - 1] = '\0';
  make_toks(buff, args);
  if (args[0] == NULL)
   continue;
  if (strcmp(args[0], "quit") == 0)
   break;
  else if (strcmp(args[0], "count") == 0)
  {
   if (args[1] == NULL || args[2] == NULL)
    printf("Usage: count c|w|l filename\n");
   else
    count(args[2], args[1][0]);
  }
```

```
  else
  {
   pid = fork();
   if (pid > 0)
    wait(NULL);
   else
   {
    if (execvp(args[0], args) == -1)
     printf("Bad command\n");
    exit(1);
   }
  }
 }
 return 0;
}
```

Sample Input / Output

```
myshell$ count c test.txt
No.of characters:39
myshell$ count w test.txt
No.of words:8
myshell$ count l test.txt
No.of lines:3
myshell$ ls test.txt
test.txt
myshell$ xyz
Bad command
myshell$ quit
```

Shell style follows your sample (myshell$ prompt, make_toks, count, fork/wait/execvp). Create a small test.txt before running. Word count is by spaces and newlines, as in the sample.

SLIP 3 — Solve Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the SSTF algorithm. Requests: 98, 183, 37, 122, 14, 124, 65. Start head position: 53.

Program: slip3.c

Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, blocks, head, i, j, total = 0, served = 0;
    int req[50], done[50] = {0};

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    while (served < n) {
        int min = 100000, idx = -1;
        for (j = 0; j < n; j++) {
            if (!done[j] && abs(req[j] - head) < min) {
                min = abs(req[j] - head);
                idx = j;
            }
        }
        done[idx] = 1;
        total += min;
        head = req[idx];
        printf(" -> %d", head);
        served++;
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 98 183 37 122 14 124 65
Enter current head position: 53

Order of service: 53 -> 65 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total head movements = 232
```

SLIP 4 — Solve Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the SCAN algorithm. Requests: 82, 170, 43, 140, 24, 16, 190, 65. Starting head position 50, direction Left.

Program: slip4.c

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        go(0);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        go(blocks - 1);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 8
Enter request string: 82 170 43 140 24 16 190 65
Enter current head position: 50
Enter direction (0 = Left, 1 = Right): 0

Order of service: 50 -> 43 -> 24 -> 16 -> 0 -> 65 -> 82 -> 140 -> 170 -> 190
Total head movements = 240
```

SLIP 5 — Solve Q1 (15 marks)

Question: Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the FIFO page replacement algorithm. Reference string: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6.

Program: slip5.c

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], n, f, i, j, faults = 0, hit, next = 0;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                break;
            }
        if (!hit) {
            frames[next] = ref[i];
            next = (next + 1) % f;
            faults++;
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 4 5 6 3 4 7 3 4 5 6 7 2 4 6

Ref     Frames          Status
3       3 - -           Fault
4       3 4 -           Fault
5       3 4 5           Fault
6       6 4 5           Fault
3       6 3 5           Fault
4       6 3 4           Fault
7       7 3 4           Fault
3       7 3 4           Hit
4       7 3 4           Hit
5       7 5 4           Fault
6       7 5 6           Fault
7       7 5 6           Hit
2       2 5 6           Fault
4       2 4 6           Fault
6       2 4 6           Hit

Total page faults = 11
```

SLIP 9 — Solve Q1 (15 marks)

Question: Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the Optimal page replacement algorithm. Reference string: 8,5,7,8,5,7,2,3,7,3,5,9,4,6,2.

Program: slip9.c

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], n, f, i, j, k, faults = 0, hit, filled = 0;
    int farthest, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                frames[filled++] = ref[i];
            } else {
                farthest = -1;
                victim = 0;
                for (j = 0; j < f; j++) {
                    for (k = i + 1; k < n; k++)
                        if (frames[j] == ref[k])
                            break;
                    if (k > farthest) {
                        farthest = k;
                        victim = j;
                    }
                }
                frames[victim] = ref[i];
            }
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2

Ref     Frames          Status
8       8 - -           Fault
5       8 5 -           Fault
7       8 5 7           Fault
8       8 5 7           Hit
5       8 5 7           Hit
```

```
7       8 5 7           Hit
2       2 5 7           Fault
3       3 5 7           Fault
7       3 5 7           Hit
3       3 5 7           Hit
5       3 5 7           Hit
9       9 5 7           Fault
4       4 5 7           Fault
6       6 5 7           Fault
2       2 5 7           Fault

Total page faults = 9
```

SLIP 10 — Solve Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the LOOK algorithm. Requests: 55, 58, 39, 18, 90, 160, 150, 38. Start head position: 100, direction Left.

Program: slip10.c

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 8
Enter request string: 55 58 39 18 90 160 150 38
Enter current head position: 100
Enter direction (0 = Left, 1 = Right): 0

Order of service: 100 -> 90 -> 58 -> 55 -> 39 -> 38 -> 18 -> 150 -> 160
Total head movements = 224
```

SLIP 11 — Solve Q2 (15 marks)

Question: Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the Optimal page replacement algorithm. Reference string: 8,5,7,8,5,7,2,3,7,3,5,9,4,6,2. (Repeated: same as Slip 9 Q1)

Program: slip11.c

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], n, f, i, j, k, faults = 0, hit, filled = 0;
    int farthest, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                frames[filled++] = ref[i];
            } else {
                farthest = -1;
                victim = 0;
                for (j = 0; j < f; j++) {
                    for (k = i + 1; k < n; k++)
                        if (frames[j] == ref[k])
                            break;
                    if (k > farthest) {
                        farthest = k;
                        victim = j;
                    }
                }
                frames[victim] = ref[i];
            }
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2

Ref     Frames          Status
8       8 - -           Fault
5       8 5 -           Fault
7       8 5 7           Fault
8       8 5 7           Hit
```

```
5       8 5 7           Hit
7       8 5 7           Hit
2       2 5 7           Fault
3       3 5 7           Fault
7       3 5 7           Hit
3       3 5 7           Hit
5       3 5 7           Hit
9       9 5 7           Fault
4       4 5 7           Fault
6       6 5 7           Fault
2       2 5 7           Fault

Total page faults = 9
```

SLIP 12 — Solve Q2 (15 marks)

Question: Implement a C program to accept n integers to be sorted. Main function creates a child process using fork. Parent sorts the integers using bubble sort and waits for the child using wait; child sorts the integers using insertion sort.

Program: slip12.c

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void show(int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int a[50], b[50], n, i, j, t, key, pid;

    printf("Enter number of integers: ");
    scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        b[i] = a[i];
    }
    fflush(stdout);

    pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
        exit(1);
    } else if (pid == 0) {
        for (i = 1; i < n; i++) {
            key = b[i];
            j = i - 1;
            while (j >= 0 && b[j] > key) {
                b[j + 1] = b[j];
                j--;
            }
            b[j + 1] = key;
        }
        printf("Child  (insertion sort): ");
        show(b, n);
    } else {
        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) {
                    t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
        printf("Parent (bubble sort)   : ");
        show(a, n);
        wait(NULL);
        printf("Child process completed.\n");
    }
    return 0;
}
```

Sample Input / Output

```
Enter number of integers: 5
Enter 5 integers: 64 25 12 22 11
Parent (bubble sort)   : 11 12 22 25 64
Child  (insertion sort): 11 12 22 25 64
Child process completed.
```

SLIP 13 — Solve Q2 (15 marks)

Question: Write a simulation program for disk scheduling using the LOOK algorithm. Requests: 15, 30, 55, 90, 125, 140, 170. Starting head position: 100, direction Left. (Repeated: same program as Slip 10 Q2)

Program: slip13.c

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 15 30 55 90 125 140 170
Enter current head position: 100
Enter direction (0 = Left, 1 = Right): 0

Order of service: 100 -> 90 -> 55 -> 30 -> 15 -> 125 -> 140 -> 170
Total head movements = 240
```

SLIP 16 — Solve Q1 (15 marks)

Question: Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (counter method) page replacement algorithm. Reference string: 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2.

Program: slip16.c

Program

```
#include <stdio.h>

int main() {
    int ref[50], frames[10], last[10], n, f, i, j, faults = 0, hit;
    int filled = 0, victim;

    printf("Enter number of frames: ");
    scanf("%d", &f);
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < f; i++) {
        frames[i] = -1;
        last[i] = 0;
    }

    printf("\nRef\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        hit = 0;
        for (j = 0; j < f; j++)
            if (frames[j] == ref[i]) {
                hit = 1;
                last[j] = i + 1;
                break;
            }
        if (!hit) {
            faults++;
            if (filled < f) {
                victim = filled++;
            } else {
                victim = 0;
                for (j = 1; j < f; j++)
                    if (last[j] < last[victim])
                        victim = j;
            }
            frames[victim] = ref[i];
            last[victim] = i + 1;
        }
        printf("%d\t", ref[i]);
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\t\t%s\n", hit ? "Hit" : "Fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    return 0;
}
```

Sample Input / Output

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2

Ref     Frames          Status
3       3 - -           Fault
5       3 5 -           Fault
7       3 5 7           Fault
2       2 5 7           Fault
5       2 5 7           Hit
1       2 5 1           Fault
2       2 5 1           Hit
```

```
3       2 3 1           Fault
1       2 3 1           Hit
3       2 3 1           Hit
5       5 3 1           Fault
3       5 3 1           Hit
1       5 3 1           Hit
6       6 3 1           Fault
2       6 2 1           Fault

Total page faults = 9
```

SLIP 16 — Solve Q2 (alternative) (15 marks)

Question: Write a C program that behaves like a shell which displays the command prompt. It accepts the command, tokenizes the command line and executes it by creating a child process. Also implement the additional command count: count c filename (characters), count w filename (words), count l filename (lines).

Program: slip16.c

Program

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

void make_toks(char *s, char *tok[])
{
 int i = 0;
 char *p;
 p = strtok(s, " ");
 while (p != NULL)
 {
  tok[i++] = p;
  p = strtok(NULL, " ");
 }
 tok[i] = NULL;
}

void count(char *fn, char op)
{
 int fh, cc = 0, wc = 0, lc = 0;
 char c;
 fh = open(fn, O_RDONLY);
 if (fh == -1)
 {
  printf("File %s not found.\n", fn);
  return;
 }
 while (read(fh, &c, 1) > 0)
 {
  if (c == ' ')
   wc++;
  else if (c == '\n')
  {
   wc++;
   lc++;
  }
  cc++;
 }
 close(fh);
 switch (op)
 {
 case 'c':
  printf("No.of characters:%d\n", cc);
  break;
 case 'w':
  printf("No.of words:%d\n", wc);
  break;
 case 'l':
  printf("No.of lines:%d\n", lc);
  break;
 }
}

int main()
{
 char buff[80], *args[10];
 int pid;
 while (1)
 {
  printf("myshell$ ");
  fflush(stdout);
  fgets(buff, 80, stdin);
  buff[strlen(buff) - 1] = '\0';
```

```
  make_toks(buff, args);
  if (args[0] == NULL)
   continue;
  if (strcmp(args[0], "quit") == 0)
   break;
  else if (strcmp(args[0], "count") == 0)
  {
   if (args[1] == NULL || args[2] == NULL)
    printf("Usage: count c|w|l filename\n");
   else
    count(args[2], args[1][0]);
  }
  else
  {
   pid = fork();
   if (pid > 0)
    wait(NULL);
   else
   {
    if (execvp(args[0], args) == -1)
     printf("Bad command\n");
    exit(1);
   }
  }
 }
 return 0;
}
```

Sample Input / Output

```
myshell$ count c test.txt
No.of characters:39
myshell$ count w test.txt
No.of words:8
myshell$ count l test.txt
No.of lines:3
myshell$ ls test.txt
test.txt
myshell$ xyz
Bad command
myshell$ quit
```

Shell style follows your sample (myshell$ prompt, make_toks, count, fork/wait/execvp). Create a small test.txt before running. Word count is by spaces and newlines, as in the sample.

SLIP 20 — Solve Q1 (15 marks)

Question: Write a simulation program for disk scheduling using the LOOK algorithm. Requests: 86, 147, 91, 177, 45, 12, 130. Starting head position: 60, direction Right. (Repeated: same program as Slip 10 Q2)

Program: slip20.c

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 86 147 91 177 45 12 130
Enter current head position: 60
Enter direction (0 = Left, 1 = Right): 1

Order of service: 60 -> 86 -> 91 -> 130 -> 147 -> 177 -> 45 -> 12
Total head movements = 282
```

SLIP 23 — Solve Q1 (15 marks)

Question: Implement a C program to accept n integers to be sorted. Main function creates a child process using fork. Parent sorts the integers using bubble sort and waits for the child using wait; child sorts the integers using insertion sort. (Repeated: same as Slip 12 Q2)

Program: slip23.c

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void show(int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int a[50], b[50], n, i, j, t, key, pid;

    printf("Enter number of integers: ");
    scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        b[i] = a[i];
    }
    fflush(stdout);

    pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
        exit(1);
    } else if (pid == 0) {
        for (i = 1; i < n; i++) {
            key = b[i];
            j = i - 1;
            while (j >= 0 && b[j] > key) {
                b[j + 1] = b[j];
                j--;
            }
            b[j + 1] = key;
        }
        printf("Child  (insertion sort): ");
        show(b, n);
    } else {
        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) {
                    t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
        printf("Parent (bubble sort)   : ");
        show(a, n);
        wait(NULL);
        printf("Child process completed.\n");
    }
    return 0;
}
```

Sample Input / Output

```
Enter number of integers: 6
Enter 6 integers: 50 30 10 40 20 60
Parent (bubble sort)   : 10 20 30 40 50 60
Child  (insertion sort): 10 20 30 40 50 60
Child process completed.
```

SLIP 25 — Solve Q1 (15 marks)

Question: Write a C program to illustrate the concept of an orphan process. Parent creates a child and terminates before the child has finished its task, so the child becomes an orphan. (Use fork(), sleep(), getpid(), getppid())

Program: slip25.c

Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    if (pid == 0) {
        printf("Child : PID = %d, Parent PID = %d\n", getpid(), getppid());
        sleep(5);
        printf("Child (now orphan): PID = %d, New Parent PID = %d\n", getpid(), getppid());
    } else {
        printf("Parent: PID = %d, Child PID = %d\n", getpid(), pid);
        printf("Parent terminating before child finishes.\n");
    }
    return 0;
}
```

Sample Input / Output

```
$ ./orphan
Parent: PID = 8120, Child PID = 8121
Child : PID = 8121, Parent PID = 8120
Parent terminating before child finishes.
$
(after about 5 seconds)
Child (now orphan): PID = 8121, New Parent PID = 1
```

Note: PIDs differ on every run. The last line appears about 5 seconds after the parent has ended; New Parent PID is usually 1 (init/systemd).

SLIP 27 — Solve Q1 (15 marks)

Question: Write a simulation program for disk scheduling using the SSTF algorithm. Requests: 30, 10, 60, 95, 120, 150, 175. Start head position: 50. (Repeated: same program as Slip 3 Q2)

Program: slip27.c

Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, blocks, head, i, j, total = 0, served = 0;
    int req[50], done[50] = {0};

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    while (served < n) {
        int min = 100000, idx = -1;
        for (j = 0; j < n; j++) {
            if (!done[j] && abs(req[j] - head) < min) {
                min = abs(req[j] - head);
                idx = j;
            }
        }
        done[idx] = 1;
        total += min;
        head = req[idx];
        printf(" -> %d", head);
        served++;
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 30 10 60 95 120 150 175
Enter current head position: 50

Order of service: 50 -> 60 -> 30 -> 10 -> 95 -> 120 -> 150 -> 175
Total head movements = 225
```

SLIP 28 — Solve Q1 (15 marks)

Question: Write a simulation program for disk scheduling using the SCAN algorithm. Requests: 10, 25, 75, 90, 130, 145, 180, 55. Starting head position: 80, direction Left. (Repeated: same program as Slip 4 Q2)

Program: slip28.c

Program

```
#include <stdio.h>
#include <stdlib.h>

int cur, total = 0;

void go(int x) {
    printf(" -> %d", x);
    total += abs(x - cur);
    cur = x;
}

int main() {
    int req[50], n, blocks, head, dir, i, j, t, pos;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request string: ");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter current head position: ");
    scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): ");
    scanf("%d", &dir);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) {
                t = req[j];
                req[j] = req[j + 1];
                req[j + 1] = t;
            }

    for (pos = 0; pos < n && req[pos] < head; pos++);

    cur = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
        go(0);
        for (i = pos; i < n; i++)
            go(req[i]);
    } else {
        for (i = pos; i < n; i++)
            go(req[i]);
        go(blocks - 1);
        for (i = pos - 1; i >= 0; i--)
            go(req[i]);
    }
    printf("\nTotal head movements = %d\n", total);
    return 0;
}
```

Sample Input / Output

```
Enter total number of disk blocks: 200
Enter number of requests: 8
Enter request string: 10 25 75 90 130 145 180 55
Enter current head position: 80
Enter direction (0 = Left, 1 = Right): 0

Order of service: 80 -> 75 -> 55 -> 25 -> 10 -> 0 -> 90 -> 130 -> 145 -> 180
Total head movements = 260
```